Error Detection and Correction at Data Link Layer using Hamming Code

Exp.no : 6

Date : 06/8/2096

Aim :

To write a program to implement error detection and correction using the Hamming code.

Algorithm :

1.) Start the program

2.) Read the number of 4 bits data blocks (n) forming the input data stream.

3.) For each 4 bit data block D = d_1 d_2 d_3 d_4, perform Hamming encoding :-

a.) Arrange 7 bits with parity bits at positions 1, 2 and 4 and data bits at positions 3, 5, 6 and 7.

b.) Compute the parity bits using even parity

$P_1 = d_1$ XOR d_2 XOR d_4 (covers bit positions 1, 3, 5, 7)

$P_2 = d_1$ XOR d_3 XOR d_4 (covers bit positions 2, 3, 6, 7)

$P_4 = d_2$ XOR d_3 XOR d_4 (covers bit positions 4, 5, 6, 7)

c.) Form the 7-bit Hamming code word :

P_1 P_2 d_1 P_4 d_2 d_3 d_4.

4.) Simulate transmission by introducing a single-bit error at a chosen bit position (entered by the user) in the code word.

Scanned with OKEN Scanner

5.) At the receiver, recompute the check bits (syndrome) from the received code word

$c_1 = p_1 \text{ XOR } d_1 \text{ XOR } d_2 \text{ XOR } d_4$

$c_2 = p_2 \text{ XOR } d_1 \text{ XOR } d_3 \text{ XOR } d_4$

$c_3 = p_4 \text{ XOR } d_2 \text{ XOR } d_3 \text{ XOR } d_4$

6.) Compute Error position = $c_4\, c_2\, c_1$ (binary number formed from the check bits)

7.) If Error position = 0, no error has occurred. Else, the bit at "Error position" is inverted (complemented) to correct the error.

8.) Extract the original 4 data bits from the ~~records~~ corrected 7 bit code word.

9.) Repeat steps 3 to 8 for all n blocks in the data stream.

10.) Stop the program

program

```
def encode(d):
    """d: list of 4 bits [d1, d2, d3, d4] → returns
    7 bit hamming code list positions (1-indexed):
    1=p1  2=p2  3=d1  4=p4  5=d2  6=d3  7=d4"""
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
```

```
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f" Error detected at bit position: {error_pos}
            -> correcting...")
        h[error_pos-1] ^= 1
    else:
        print(" No error detected.")
    return error_pos
def to_str(bits):
    return "".join(str(b) for b in bits)
def main():
    print("=== Hamming Code - Error Detection and Correction
        ===\n")
    n = int(input("Enter number of 4-bit data blocks
        in the stream: "))
    for bin range(1, n+1):
        print(f"\n--- Block {bi} ---")
```

Scanned with OKEN Scanner

```
d = list(map(int, input("Enter 4 data bits (space separated, e.g 1 0 1 1): ").split()))
h = encode(d)
print(f"Original Data Bits : {to_str(d)}")
print(f"Encoded Hamming Code (7 bits): {to_str(h)}")
pos = int(input("Enter bit position to introduce error (1-7, 0 = no error): "))
if 1 <= pos <= 7:
    h[pos-1] ^= 1
    print(f"Transmitted Code (with error included at pos {pos}): {to_str(h)}")
else:
    print(f"Transmitted Code (no error induced): {to_str(h)}")
print(f"Received Code : {to_str(h)}")
print("Checking for errors...")
detect_and_correct(h)
print(f"Corrected Code : {to_str(h)}")
rd = [h[2], h[4], h[5], h[6]]
print(f"Recovered Data Bits : {to_str(rd)}")
if __name__ == "__main__":
    main()
```

Scanned with OKEN Scanner

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits     : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code          : 0100011
Checking for errors...
  Error detected at bit position: 3 -> Correcting...
Corrected Code         : 0110011
Recovered Data Bits    : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 0
Original Data Bits     : 0110
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code          : 1100100
Checking for errors...
  Error detected at bit position: 6 -> Correcting...
Corrected Code         : 1100110
Recovered Data Bits    : 0110
```

Result :

The program to implement error detection and correction using the hamming code concept was successfully written in python, executed and verified using a sample input data stream.

Scanned with OKEN Scanner